

Presentation 2

By Akshay Anand, Ben Pilnick, Isabella Toth, and
Zoe Lively

2026

Winter Quarter

Experimental Changes to Improve Model Accuracy/Runtime (1,2,3,4,8,10)

1. Mandatory Changes
2. Variable Patch Size
3. Optuna Optimization
4. Results

Mandatory Changes

1. KimiaNet
 - a. Downloaded KimiaNet weights from [GitHub](#) for pytorch.
 - b. Modified [model.py](#) methods *init()* and *create_model()*.
 - i. Get rid of pretrained = True, replaced it with False.
 - ii. Added: state_dict.

```
torch.load("/home/bdp1083/Code1_5_ModelTraining_9Feb_Ben/KimiaNet.pth",map_location=torch.device("cpu"))  
base_model.load_state_dict(state_dict, strict=False)
```
2. Removed Saturation Jitter
3. AdamW
4. Persistent workers = True

Replacing Resizing All Images to 224x224 with Variable Patch Sizes

Method

1. **Disable patch resizing:** Allow natural patch dimensions
2. **Filter small patches:** Remove patches < 32x32 pixels
3. **Individual processing:** Loop through each patch separately

Implementation

1. Dataset Modification ([dataset.py](#)) → `load_slice_tensor()`
 # Filter during image loading
 if `img.size[0] < 32` or `img.size[1] < 32`:
 `continue` # Skip patches that are too small
 ... Rest of Function. Returns a list `List[Tensor(C,H,W)]`
2. Model Modification ([models.py](#)) → `process_single_stain()`
 Added loop for `slice_data` in `slice_list` that process each patch individually.

Variable Sized Patch → CNN feature extractor → pooled to fixed size (1024) → projected to embedding (512) → concatenated into a matrix for the slice.

These patch embeddings are then used to calculate patch attention weights and slice embeddings.

Pseudocode: New Patch Processing Loop

OLD (couldn't handle variable sizes):

```
FOR slice_tensor IN slice_list:
    patch_features = CNN(slice_tensor) # Fails with variable
    # sizes. The input expects a tensor but gets a list from l
    oad_slice_tensor.
```

NEW (handles variable sizes):

```
FOR slice_data IN slice_list:
    patch_embeddings_list = []
    FOR EACH patch IN slice_data: #Process individually
        patch = add_batch_dim(patch) # (C,H,W) → (1,C,H,W)
        features = CNN(patch) # (1,1024,h,w)
        pooled = adaptive_pool(features) # (1,1024) [fixed!]
        embedding = project(pooled) # (1,512)
        patch_embeddings_list.append(embedding)
    patch_embeddings = concat(patch_embeddings_list) # (P,512)
    slice_embedding = attention(patch_embeddings) # (512)
```

Key Change: Added inner loop to process patches one-by-one, enabling variable dimensions. `slice_list` is a list of tensors instead of a stacked tensors.

OPTUNA Optimization

Key Concepts:

1. Trial → a trial is a set of hyperparameter combos
2. Study → a collection of trials that minimizes the loss function
3. Objective Function → Gets different hyperparameter combinations from trials and trains the model. Returns the validation loss.

Implementation/Settings:

- **Optuna_training.py/run_optuna.sbatch**
- 10 trials, 8 Epochs within each trial
- TPESampler → decides what values to try next.
- Pruning → stops unpromising trials
 - N_startup_trials = 4
 - Ensures that OPTUNA cannot prune the first four trials. This is so that the model has time to learn from each trial.
 - Pruning_warmup = 3
 - Within each trial OPTUNA cannot stop the trial early before the first 3 epochs.
- Tuning 6 hyperparameters:

Learning Weight	Weight Decay	Dropout	Class Weight	Scheduler Patience	Scheduler factor
[1e-4, 5e-4]	[5e-5, 5e-4]	[0.2, 0.5]	[1.5, 3.5]	[2, 5]	[0.2, 0.5]

Ben

Zoe

Akshay

Isabella

Optuna Optimization Parameters

Code	Learning Weight	Weight Decay	Dropout	Class Weight	Scheduler Patience	Scheduler feature	Val Loss	Val Accuracy
Original code	2.6e^-4	4.1e^-4	0.22	1.89	2	0.29	0.55	0.70
No Variable Patches	2.08e^-4	3.05e^-4	0.26	2.53	4	0.21	0.36	0.77
Variable Patches								

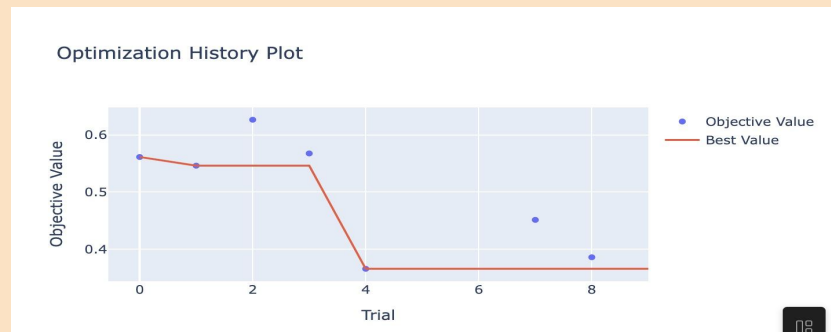
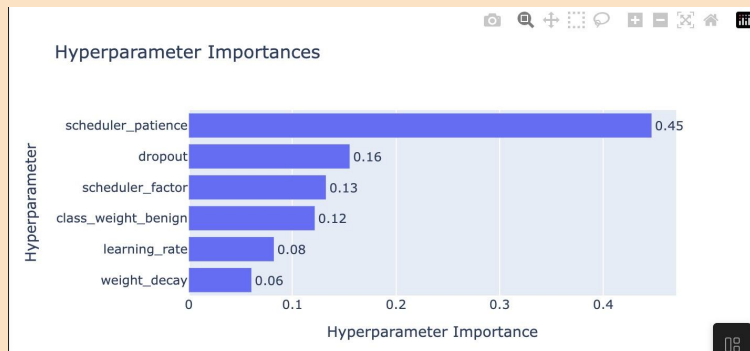
Ben

Zoe

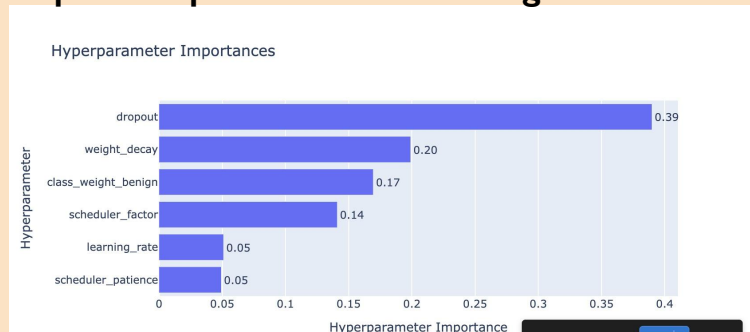
Akshay

Isabella

Optuna Optimization No Variable Patches



Optuna Optimization with Original Code



Ben

Zoe

Akshay

Isabella

Results

Type of Code	Optuna Optimization			No Optuna Optimization		
	Test Loss	*Validation Loss	Test Accuracy	Test Loss	Validation Loss	Test Accuracy
Original Code	0.78	0.56	0.67	0.73	0.45	0.78
No Variable Patches	0.84	0.39	0.83	0.50	0.42	0.83
Variable Patches				0.80	0.46	0.61

Takeaways

1. Variable Patches hurt generalization.
 - a. Test Accuracy = 0.61 while test loss = 0.80.
2. No Variable Patches performed the best.
 - a. Test Accuracy = 0.83, while test loss = 0.50
3. Optuna Optimization did not help with performance
 - a. Overfit on the training/validation set $\rightarrow$ validation loss significantly $<$ test loss.

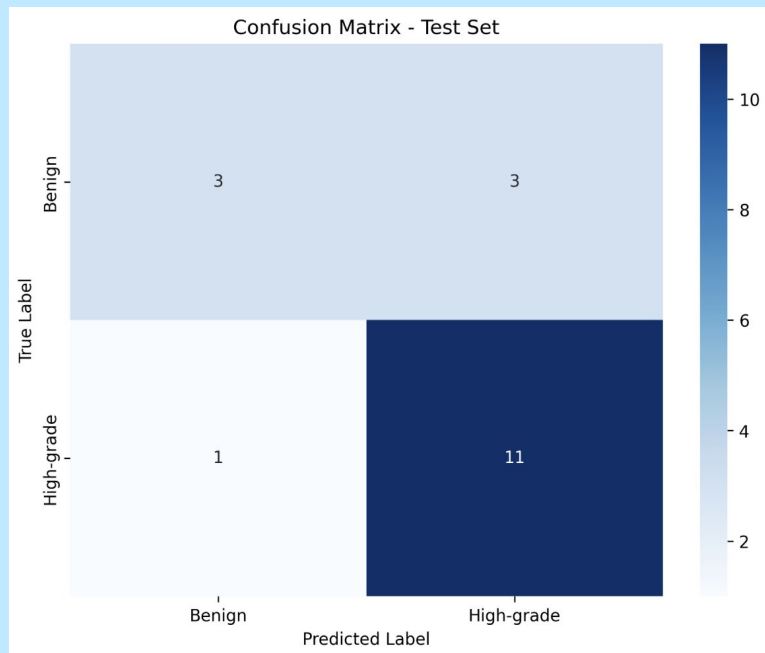
Next Steps

1. Only tune learning rate, weight decay, and dropout. Current configuration may be overfitting to the validation set too much $\rightarrow$ Hyperparameter overfitting.
2. Report results of OPTUNA Optimization from Variable patches [Not done training].
3. Implement RandomResize() and compare performance to variable patches.

Experimental Changes to Improve Model Accuracy (5 & 6)

1. Original Code
2. Just entropy regulation
3. Just gated attention
4. Both together

Original Code Results



No changes made to code from Fall 2025, just running to compare results to control

From /projects/e32998/MIL_training/final_runs/run_20260214_141512

Test Results:

- Test Loss: 0.7272
- Test Accuracy: 0.7778
- Number of samples: 18

Entropy Regularization

- **Proposal 2:** entropy regularization (*From Final Presentation Slides*)
 - For each attention layer, Add penalty term based on entropy after softmax
 - Entropy formula
$$H(w) = - \sum_i w_i \log w_i$$
- - If attention weights are more uniformly distributed, entropy is high
 - If we have very sparse attention distribution, entropy is low
 - Penalty = $-\lambda H(w)$
 - This gives more penalty when entropy is low. In gradient descent, this pushes smaller attention weight up, and large attention weight down. aka penalize “large attention larger, smaller attention smaller”
 - Ideas about tuning:
 - Larger λ -> stability; smaller λ -> selectivity
 - Similar idea: gradually decrease λ as we moves from patch attention layer to stain layer (so that later layer are more selective)

ER Code Changes – config.py

Added a new hyperparameter: **entropy_lambda = 1e⁻³** in TRAINING_CONFIG()

- Controls strength of entropy regularization
- $\lambda = 0 \rightarrow$ no effect
- $\lambda > 0 \rightarrow$ encourages higher-entropy attention
- To prevent attention weights from collapsing onto 1-2 patches

ER Code Changes – trainer.py

1. New function added: `def attention_entropy(w):`

- a. Computes entropy

$$H(w) = - \sum_i w_i \log w_i$$

- b. Applied after softmax, averaged across attention dimension and across attention levels

2. Training loop modified

- a. Before: `Loss = CrossEntropy`
- b. After: `CrossEntropy -  $\lambda$  · H(w)`
- c. Applied to case-, slice-, and patch-level attention
- d. Encourages broader attention dist, less peaky weights, improved robustness
- e. Doesn't affect model architecture, forward pass, evaluation
 - i. Only affects training objective

```
def attention_entropy(w: torch.Tensor, eps: float = 1e-12) -> torch.Tensor:
    """
    Entropy of attention weights AFTER softmax.
    w can be shape (... , M). Returns mean entropy (scalar).
    """
    # Softmax outputs should already be >0,
    # but eps prevents log(0) if something odd happens.
    w = w.clamp_min(eps)
    H = -(w * w.log()).sum(dim=-1) # sum over attention dimension
    return H.mean()
```

ER Code Changes – trainer.py

Inside def_train_epoch():

Before:

```
outputs = self.model(stain_slices)
outputs = outputs.unsqueeze(0)

base_loss = self.criterion(outputs, label)
loss = base_loss
```

After:

```
lam = float(TRAINING_CONFIG.get("entropy_lambda", 0.0))

# ----- Forward pass -----
if lam > 0.0:
    outputs, attn = self.model(
        stain_slices,
        return_attn_weights=True
    )
else:
    outputs = self.model(stain_slices)
    attn = None

outputs = outputs.unsqueeze(0)

if label.dim() == 0:
    label = label.unsqueeze(0)

base_loss = self.criterion(outputs, label)
```

ER Code Changes – trainer.py

Inside `def_train_epoch()`:

after `base_loss`, added:

```
entropies = []

if lam > 0.0:
    entropies.append(attention_entropy(attn["case_weights"]))

    for stain, d in attn.get("stain_weights", {}).items():
        if "slice_weights" in d:
            entropies.append(attention_entropy(d["slice_weights"]))

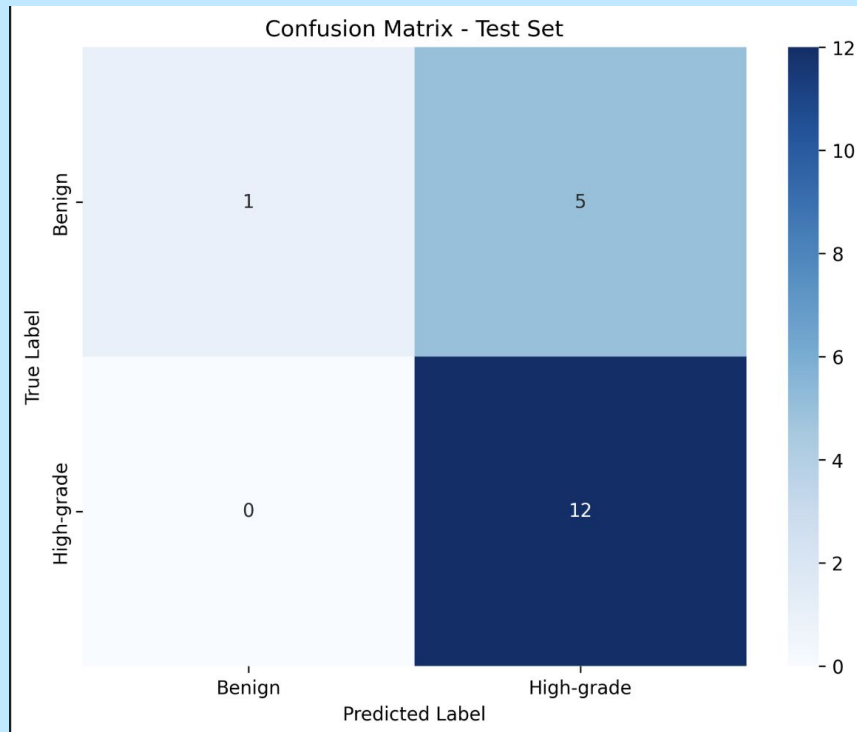
        for pw in d.get("patch_weights", []):
            entropies.append(attention_entropy(pw))

if len(entropies) > 0:
    entropy_term = torch.stack(entropies).mean()
else:
    entropy_term = torch.tensor(0.0, device=self.device)
```

Final loss change:

$$\text{loss} = \text{base_loss} - \text{lam} * \text{entropy_term}$$

ER Results



Test Results:

- Test Loss: 0.6369 (from 0.73)
- Test Accuracy: 0.7222 (from 0.78)
- Number of samples: 18

Observations:

- High-grade recall: 100% (12/12)
- No high-grade cases missed
- Increased tendency to classify as high-grade
- Benign recall decreased

ER Code Result Takeaways

Why didn't accuracy increase?

- Small test set ($n = 18$) $\rightarrow$ 1-2 prediction changes significantly affect accuracy
- With argmax thresholding, the ER run predicts high-grade for 17/18 test cases (bias toward high-grade).
- Entropy regularization shapes attention distributions – it does not directly optimize accuracy

Key Takeaways

- High-grade sensitivity 100% on this test set (12/12 detected)
- Test loss decreased ($0.73 \rightarrow 0.64$), suggesting improved calibration
- ER altered model behavior without changing architecture
- Larger dataset needed to assess consistent accuracy gains
- Can play around with $1e-3$ value to tune

Gated Attention – models.py

Old attention (baseline):

1. Single projection
2. Tanh is approximately linear in $(-1, 1)$
3. Attention scoring behaves nearly linear

New gated attention:

1. Two nonlinear branches (tanh + sigmoid)
2. Multiplicative gating
3. Increased expressiveness
4. Allows nonlinear modeling of instance importance

$$a = \text{softmax}(w^T \tanh(Vx))$$

$$a = \text{softmax}(w^T (\tanh(Vx) \odot \sigma(Ux)))$$

Gated Attention – models.py (con't)

Code changes:

Added:

```
self.V = nn.Linear(...)
```

```
self.U = nn.Linear(...)
```

```
self.w = nn.Linear(...)
```

Removed:

```
self.attention = nn.Sequential(...)
```

Gated attention is more expressive, and can model nonlinear instance importance. Doesn't change DenseNet121, 3-level hierarchy, training loop, and loss function. Only attention scoring function changed.

GA Code Changes – models.py

Inside class AttentionPool():

In `__init__`, replaced:

```
self.attention = nn.Sequential(  
    nn.Linear(D, H),  
    nn.Tanh(),  
    nn.Linear(H, 1)  
)
```

with:

```
self.V = nn.Linear(in_dim, hidden_dim)  
self.U = nn.Linear(in_dim, hidden_dim)  
self.w = nn.Linear(hidden_dim, 1)
```

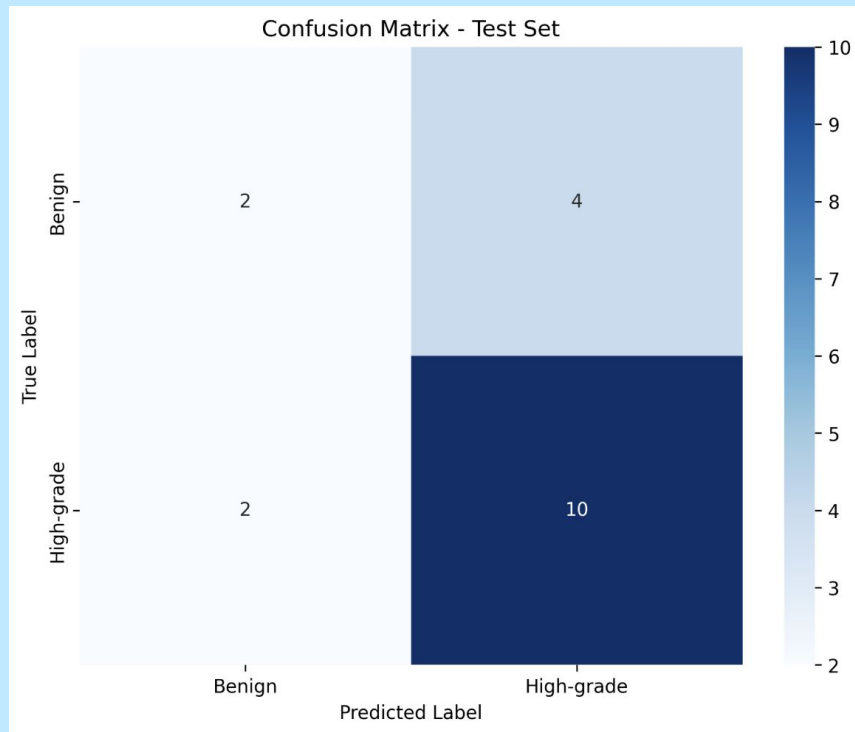
And in `def forward()`, replaced:

```
scores = self.attention(x)
```

with:

```
v = torch.tanh(self.V(x))      # (B, M, H)  
u = torch.sigmoid(self.U(x))   # (B, M, H)  
  
vu = self.dropout(v * u)      # elementwise gating  
  
scores = self.w(vu)           # (B, M, 1)
```

GA Code Results



Test Results:

- Test Loss: 0.5637 (from 0.73)
- Test Accuracy: 0.6667 (from 0.78)
- Number of samples: 18

Observations:

- High grade recall remains strong (83%)
- Benign recall decreased

GA Code Result Takeaways

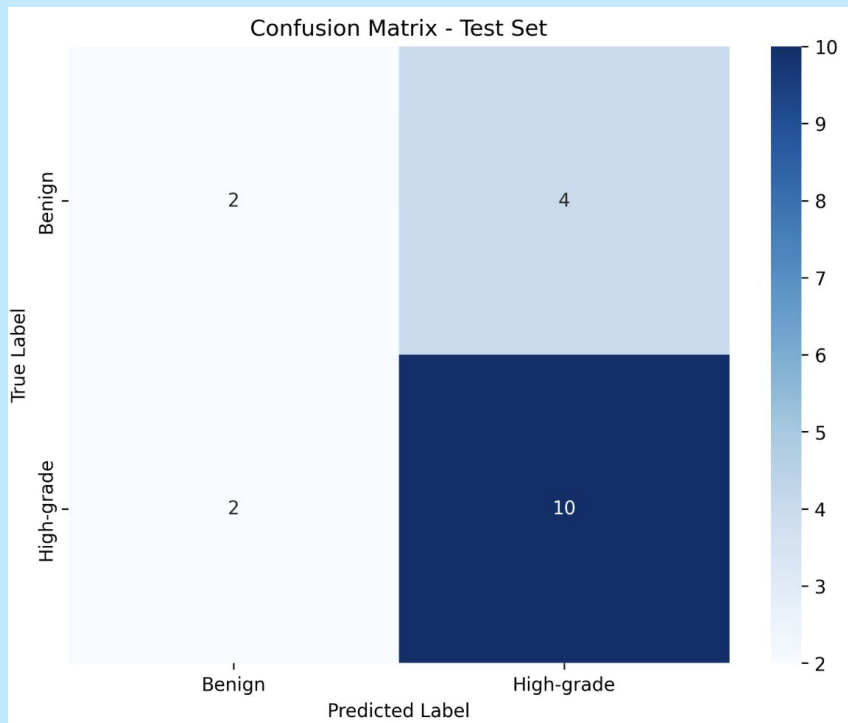
Why didn't accuracy increase?

- Small test set (18 samples) → could be worth running all 5 splits
- 2 prediction flips = 11% accuracy change
- Added model capacity does not guarantee better generalization
- Attention mechanism may not have been primary bottleneck
- Lower cross-entropy loss indicates changes in probability distribution, even though classification decisions did not improve.

Key Takeaways

- Gated attention increases expressive power
- Did not improve accuracy on this dataset
- Results within expected variance for small sample size
- Larger-scale evaluation required to assess benefit

Combo Results



Test Results:

- Test Loss: 0.5637
- Test Accuracy: 0.6667
- Number of samples: 18

Observations

- Predictions match the GA-only run on this test set (same confusion matrix + accuracy).
- High-grade recall remains 83% (10/12), benign recall remains 33% (2/6).
- Adding ER on top of GA did not change test-set decisions here (may be due to small $n=18$, λ strength, or training variance).

Combo Code Result Takeaways

Why didn't accuracy increase?

- Test set is small (n=18), so small prediction changes dominate reported accuracy.
- Combo produced the same predicted labels as GA-only on this split → ER did not measurably affect test decisions here.
- ER primarily regularizes attention distributions during training; it may not translate to higher accuracy without tuning λ / more data / repeated evaluation.

Key Takeaways

- Combo did not outperform GA-only on this test set (same accuracy, same confusion matrix).
- High-grade recall stayed relatively strong (10/12), but benign recall remained low (2/6).
- To assess benefit, run multiple seeds/splits and sweep entropy_lambda (and confirm attention weights are returned + regularized as expected).

Dropout within the model

1. Process
2. Patch Projection - first dropout
3. Attention weights at stain/slice/case level - second dropout
4. Classifier - third dropout
5. Possible changes

Process

[1] The program receives a case. It consists of

- 3 possible stains (H&E, Sox10, Melan-A)

[2] Each stain consists of a list of slices and each slice consists of a tensor of patches denoted as (P = # of patches, C = RGB channels, H = Height, W = Width)

[3] Each slice tensor is placed into Kimia Net where the tensor changes from (P, 3, 224, 224) → (P, 1024, 7, 7)

[4] The new slice tensor changed its dimensions to 2x2 with the use of Adaptive Average Pooling → (P, 1024, 2, 2)

[5] The new output is then flattened to (P, 4096)

Process cont.

Linear activation:

$$z = W_1x + b_1$$

[6] Linear activation occurs and tensor is now shifted from (P, 4096) $\rightarrow$ (P, 512)

- 512 is embed_dim hyperparameter value we set
 - Memory efficient
 - Reduces attention cost
 - Can be set to anything

Relu activation:

$$\text{ReLU}(z) = \max(0, z)$$

[7] Relu activation - Turns all values in (P, 512) vector that are less than 0, to the value 0.

First dropout occurs - 30 percent of the values within the (P, 512) are now turned to 0

- This includes values that were turned 0 in the relu activation

Process cont.

[8] Attention scores are calculated for each individual patch. It takes the 512 feature channels per patch and transforms them into 128 attention neurons

- 128 is selected from the hyperparameter attention_hidden_dim
- The attention score produces a scalar value for each patch

Second dropout occurs: 30% of the 128 attention neurons created for each of the patches is changed to 0

[9] Softmax - Each of the scalar values created for each patch is turned into a probability compared against the other scalar values. They all sum up to 1 and are measured in relative importance to one another

Important note: The scalar score for each patch is multiplied across each patch vector before summation.

[10] Weight Slice Embedding: This portion of the code collapses all the patches and sums the effect of feature 1-512 on all patches together – Think vertical collapse

Attention score equation:

$$s_i = w_2^T \tanh(W_a h_i + b_a) + b_2$$

Weight slice embedding:

$$H_{\text{slice}} = \sum_i \alpha_i h_i$$

Process cont.

[11] The attention weights and dropout at both the slice and stain level repeat the same process of what was described previously

Third dropout - occurs at the classifier stage

- Before creating our logit values from the 1x512 vector that is representative of the final learned features from all the patches, slices, and stains – dropout occurs and 30% of these values are changed to 0.

Only in training mode...

- Softmax turns the logit values into probabilities
- Cross Entropy can determine how incorrect the prediction is
- A gradient update can occur so that the model can update weights in a direction that improves loss

Cross entropy:

$$\mathcal{L}_{CE} = - \sum_k y_k \log(p_k)$$

Softmax:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Gradient Update:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Possible changes...

Currently: 30% dropout in Patch Projection, Attention Layers, and Final Classifier

- As discussed earlier, this much dropout can cause noise and affect the model's ability to learn

A possible regularization framework that could work:

- Retain/moderate dropout percentages in Patch Projection and Final Classifier
- Remove or diminish dropout percentage in Attention Layers

Implementation of tuning:

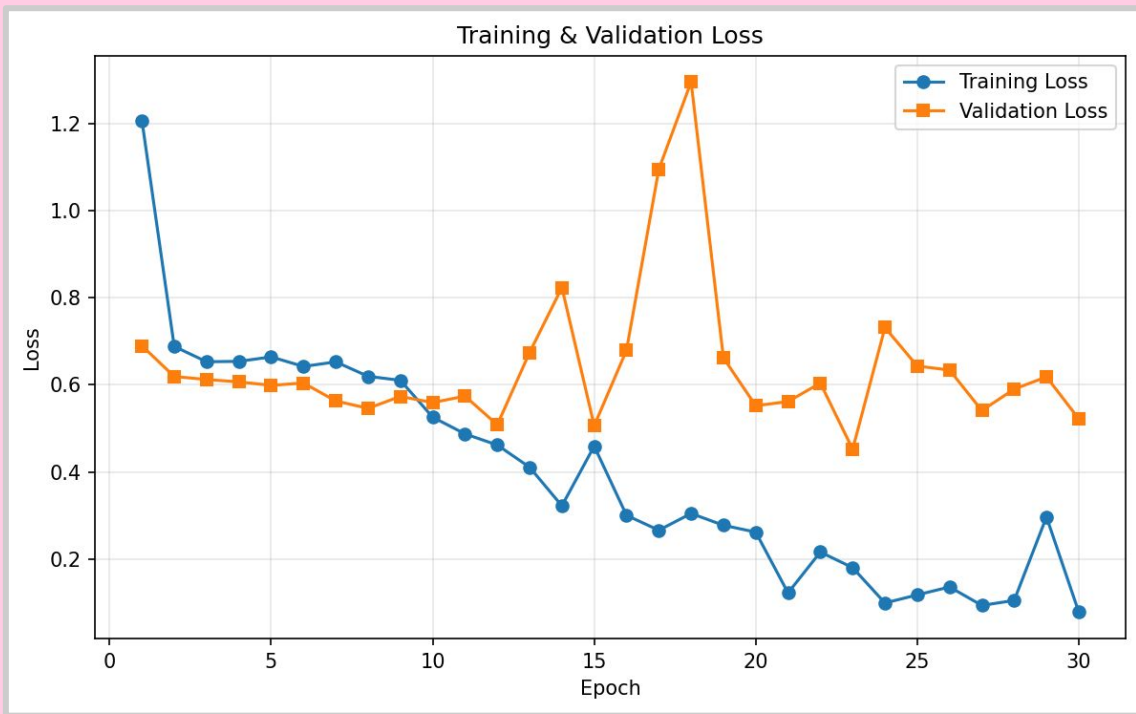
- [1] Start with small attention dropout
- [2] Monitor - Training vs Val loss and Attention Entropy value
- [3] Increase regularization if overfitting appears

Ben

Zoe

Akshay

Isabella



- Validation does not consistently decrease like training loss, erratic
 - Gap indicates overfitting
- May be too sensitive to noise in validation data
- Batch size may be too small
- Learning rate may be too high

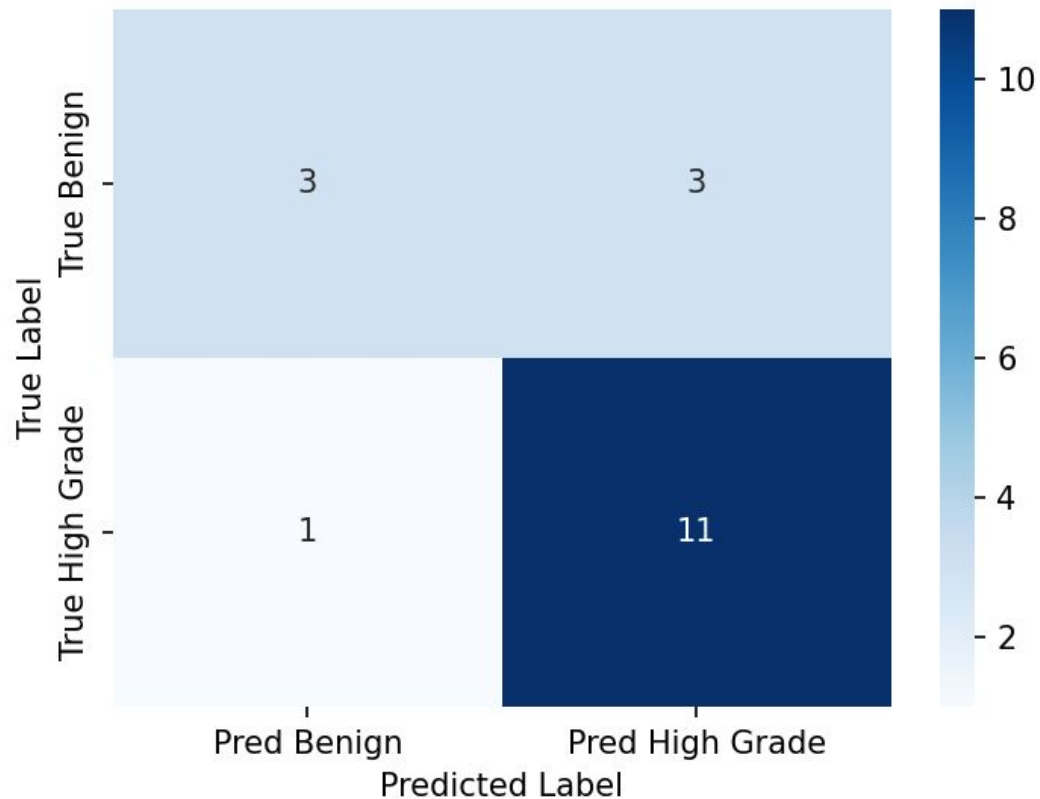
Ben

Zoe

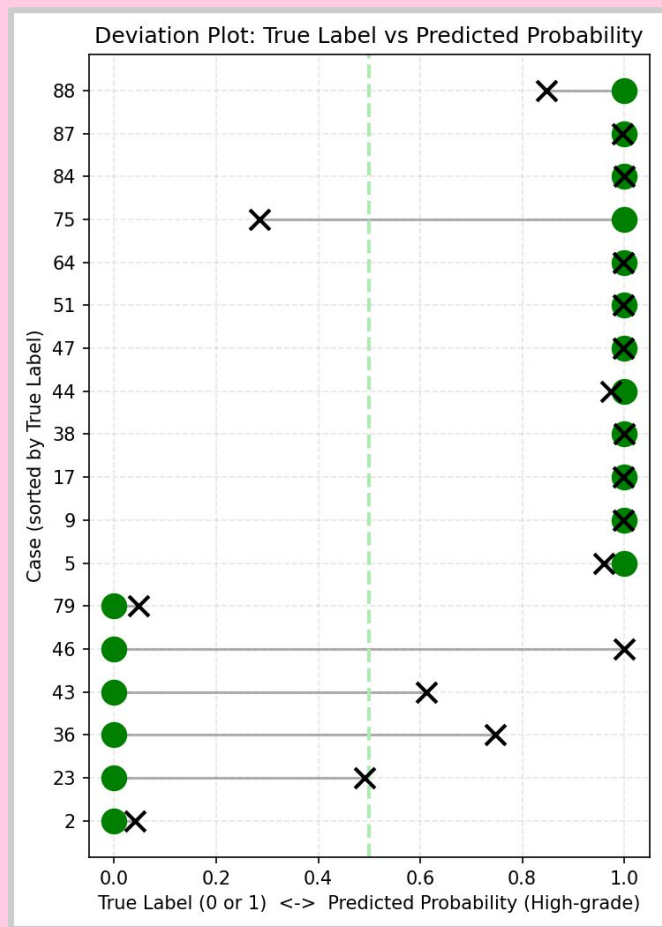
Akshay

Isabella

Confusion Matrix



- TP=11, FP=3, TN=3, FN=1 (N=18)
- Accuracy 77.8%
- Good at identifying high-grade cases



- Well-trained, not underfitting
 - Underfit models produce lots of predictions near 0.5
 - Mis-calibrated probability outputs

Concern:

- Many predictions near 1.0 → over-confident,
 - Errors are rare but when they occur they are confident errors
 - Case 46: 99.98% sure HG, but benign

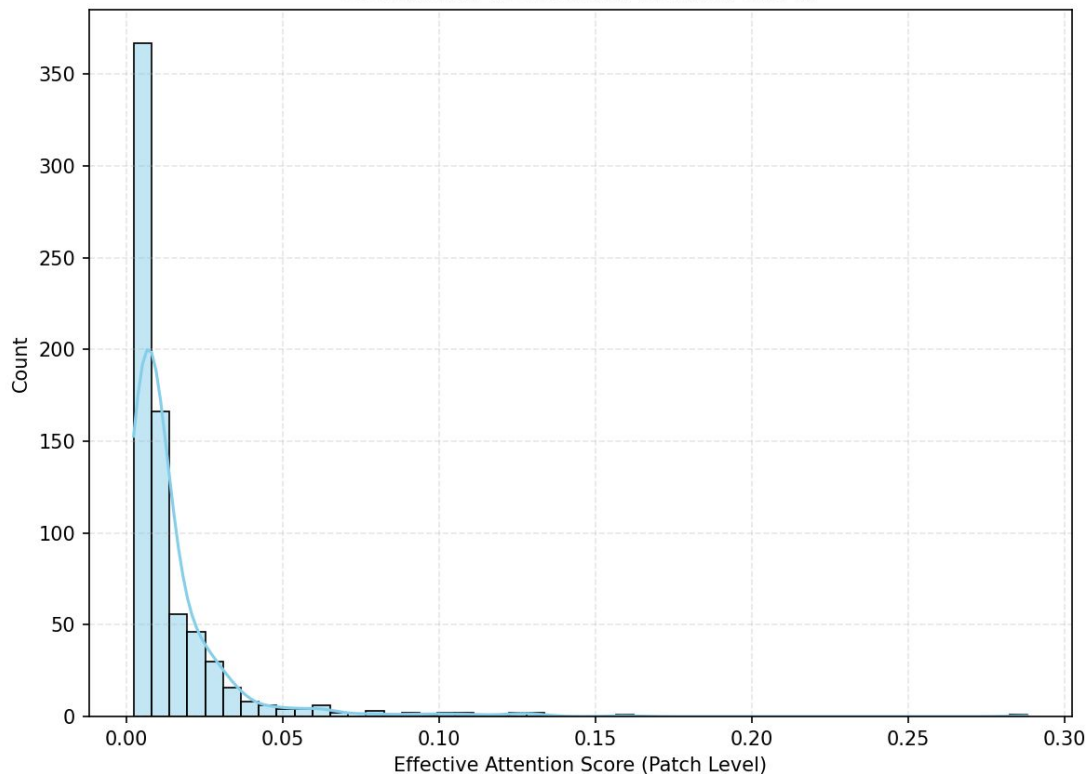
Ben

Zoe

Akshay

Isabella

Distribution of Effective Attention Scores



- Most patches have scores very close to 0- **RIGHT SKEW**
- Model focuses strongly on few patches, ignores most others

Good:

- Selective- not all patches treated equally
- Meaningful discriminative regions

Concerns:

- If important patch is noisy/missing, bad prediction
- High-attention outliers- artifacts, staining irregularities, model shortcut learning

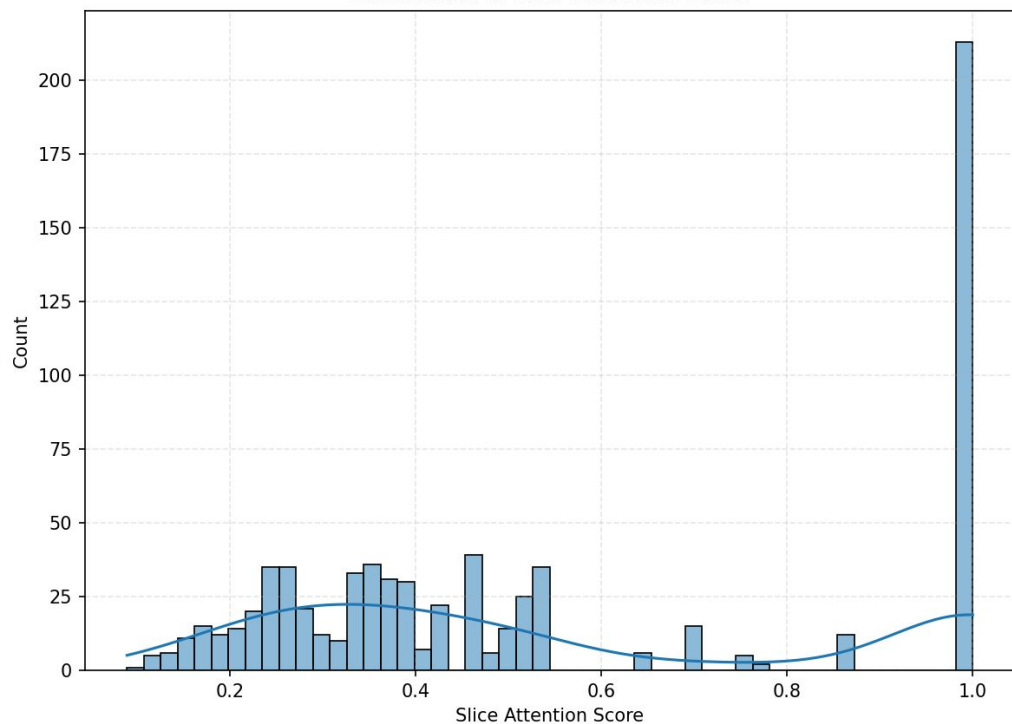
Ben

Zoe

Akshay

Isabella

Distribution of Slice Attention Scores



- Bimodal, spike at 1.0, cluster between 0.2-0.6, very few slices near 0
- Spike at 1.0 = most slices receive maximum possible attention weight
- Concentrates evidence at whole-slice level similar to patch level
 - Dependence on a single slice per case
- Small peak = sometimes multiple slices contribute, distributes attention
 - Minimal contribution

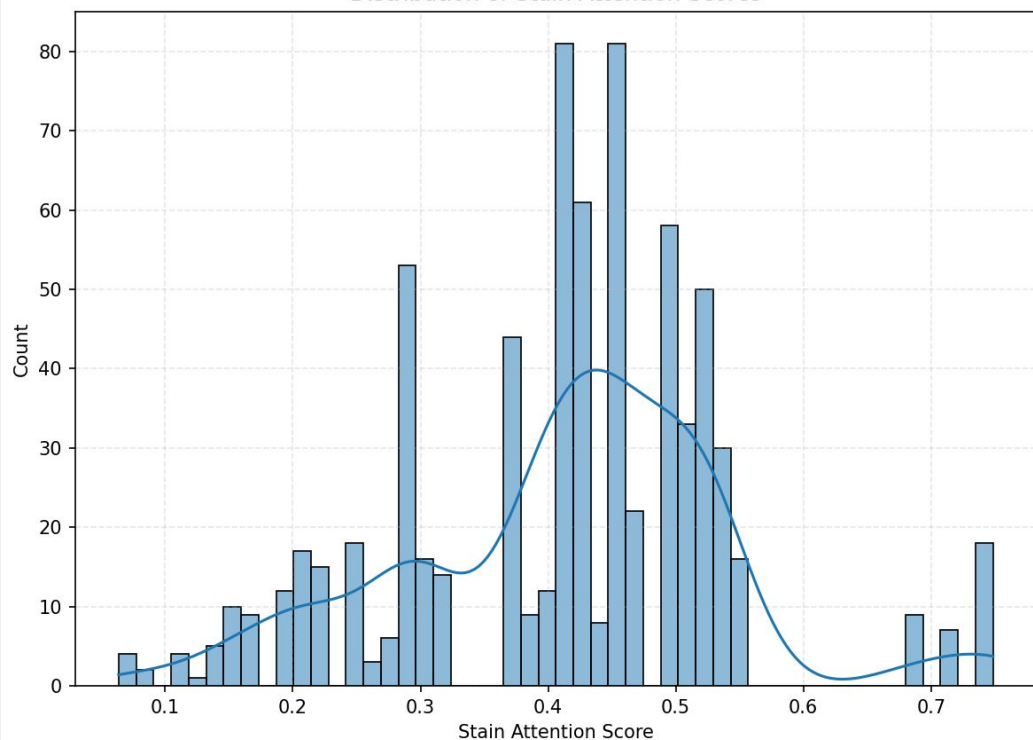
Ben

Zoe

Akshay

Isabella

Distribution of Stain Attention Scores



- Broader, multi-modal distribution
 - Some stains consistently get more attention
- Peaks around ~ 0.3 – 0.6 reflect uneven stain contributions

Concerns:

- Learning shortcuts = model may rely too much on certain stain colors or intensity differences instead of tissue structure
- High outliers = unusual outlier, harms prediction

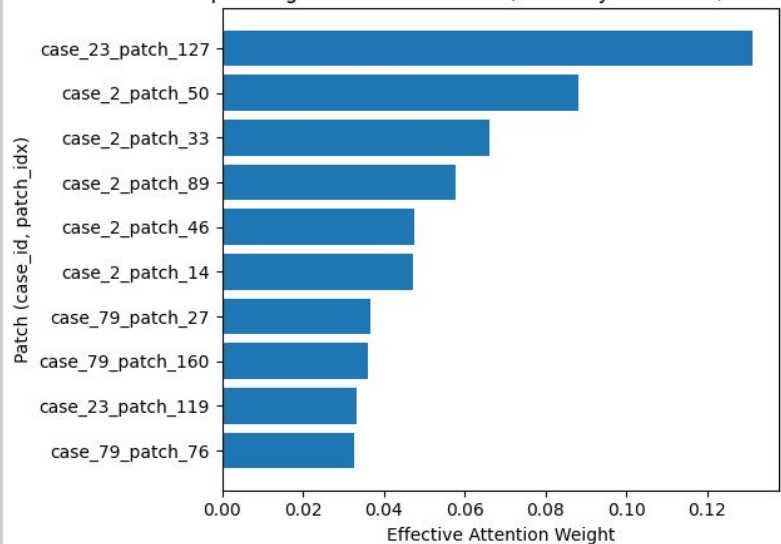
Ben

Zoe

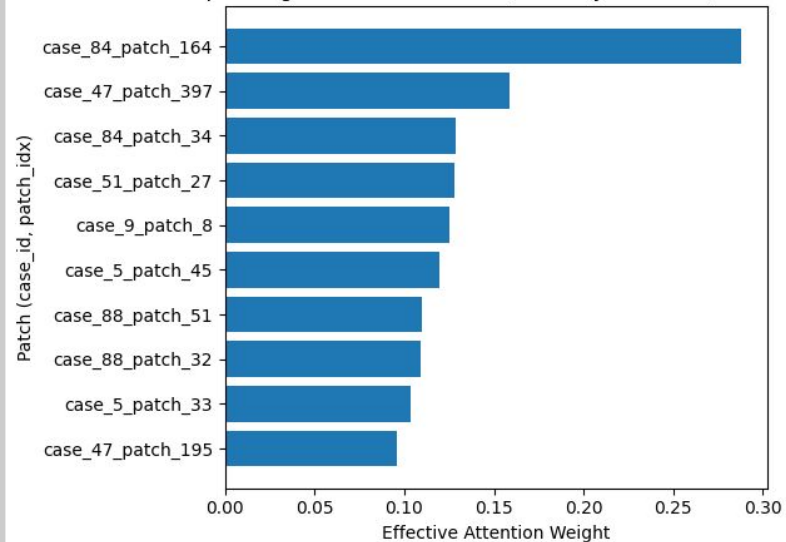
Akshay

Isabella

Top 10 High-Attention Patches (Correctly Classified, Class 0)



Top 10 High-Attention Patches (Correctly Classified, Class 1)



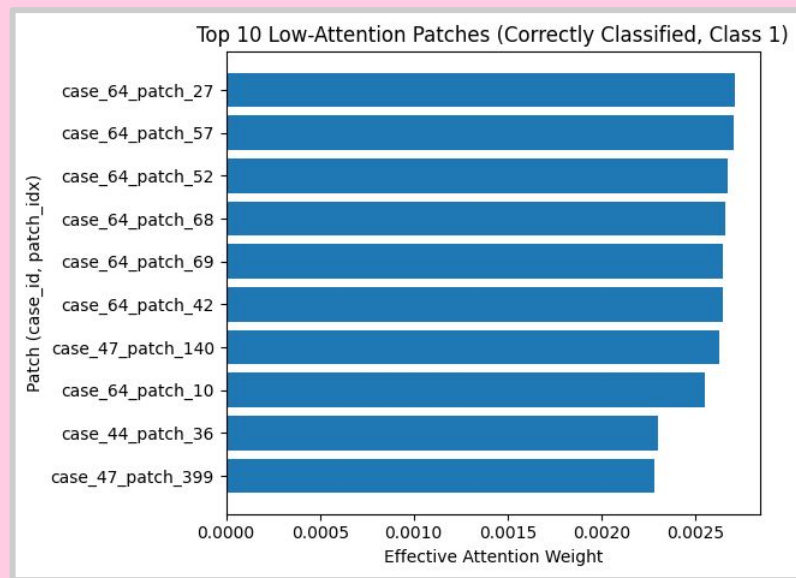
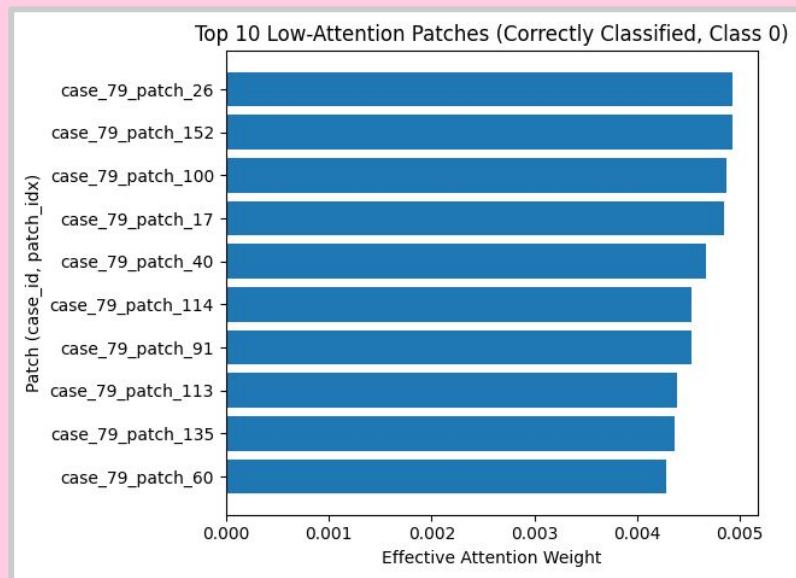
- Highest attention patches align with correct predictions
- High-grade cases show especially strong top attention weights
- Model can identify strongly discriminative regions
- Each patches indicates what model thinks is most important

Ben

Zoe

Akshay

Isabella



- Low-attention patches receive nearly zero contribution
- Model appropriately ignores irrelevant or noisy regions
- Confirms helpful sparsity when case classification is clear
- Contrasts patterns of incorrectly classified cases

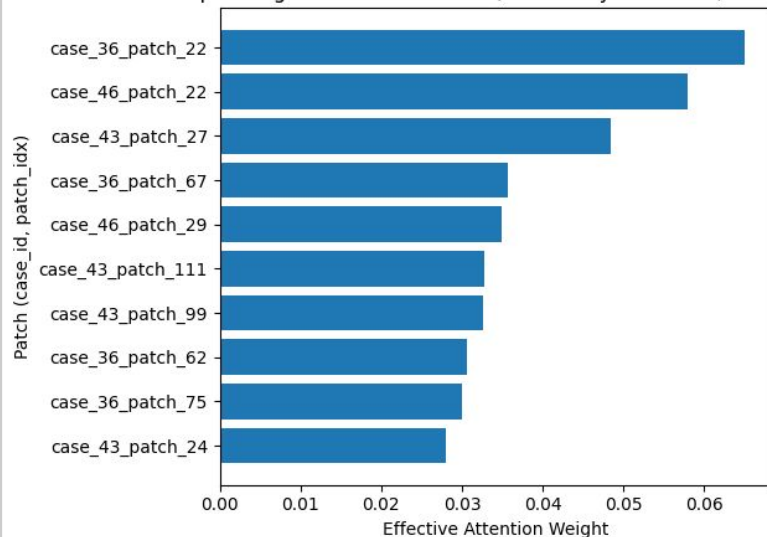
Ben

Zoe

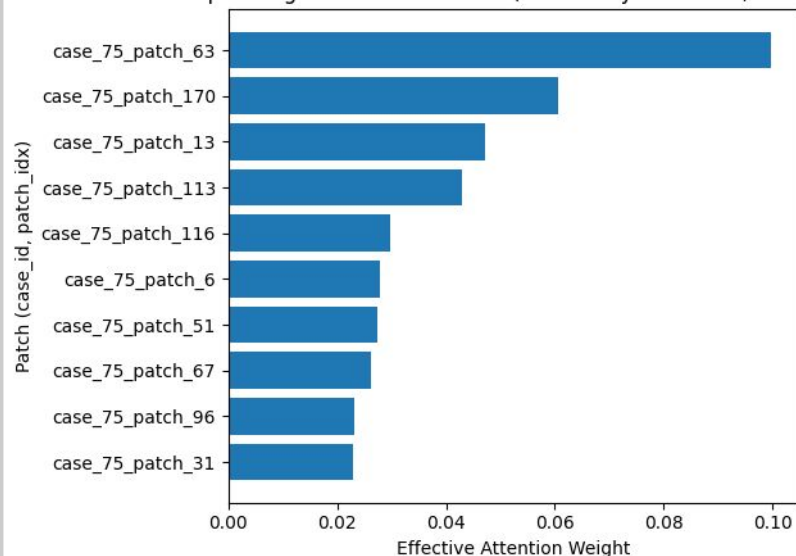
Akshay

Isabella

Top 10 High-Attention Patches (Incorrectly Classified, Class 0)



Top 10 High-Attention Patches (Incorrectly Classified, Class 1)



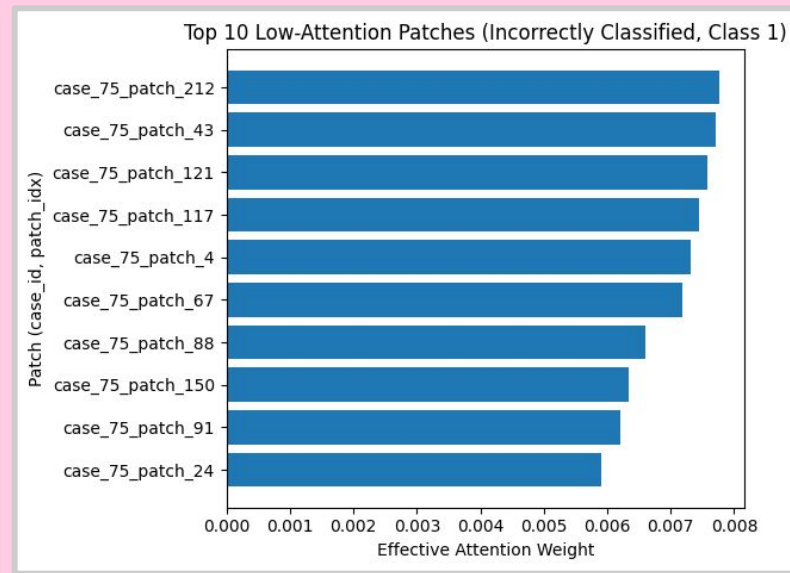
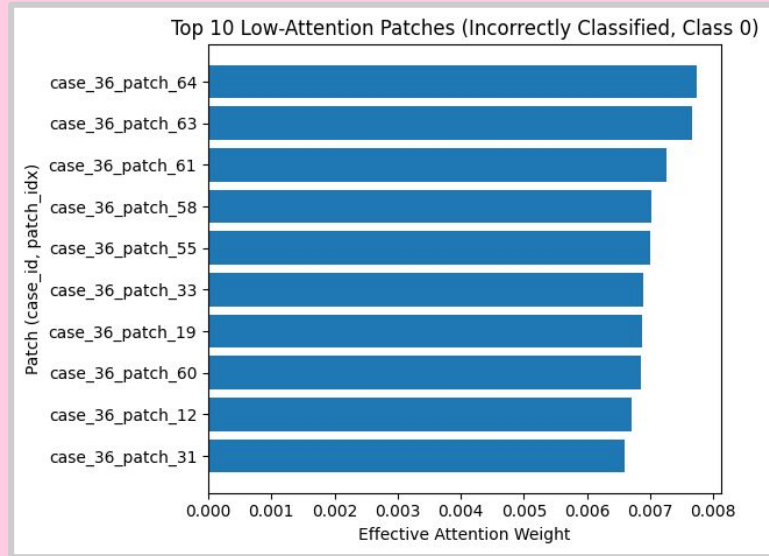
- Wrong predictions can still have highly weighted patches
- Indicates model may fixate on misleading features
- Patches often do not match true class morphology
- Useful for diagnosing why model failed

Ben

Zoe

Akshay

Isabella



- Even low-attention patches have moderate weights in misclassified cases
- Shows the model scatters attention across many patches when uncertain
- Unlike correctly classified cases, no clear discriminative region
- Reflects difficulty/unclearness of case being assessed

Next Steps

- Model Training & Generalization
 - Alternative learning rate scheduler (ReduceLROnPlateau/CosineAnnealingLN -> CosineAnnealingWarmRestarts?)
- Calibration & Decision Thresholding
 - Apply probability calibration (Platt scaling or isotonic regression)
 - Threshold sweep; improve benign specificity without losing sensitivity
- Attention Stabilization
 - Reduce attention collapse with attention-entropy regularization
 - Use top-K pooling ($K > 1$) so > 1 patch contributes to decisions
 - Add multi-head diversity losses so different heads attend to different regions
 - Adjust softmax temp/use entmax to flatten overly sharp attention peaks

Next Steps (cont.)

- Stain & Domain Robustness
 - Stain normalization (Macenko/Vahadane) to reduce stain-driven variability
 - Stain-jitter augmentation to mimic real life color differences
 - Review outliers for stains w/ high attention; check for irregularities (coloring/fading/saturation)
- Error Analysis
 - Systematically inspect high-attention patches from misclassified cases
 - Identify sources of strange attention (folds/background/inflammation/blur)
 - Add regions back to training as hard negatives to reduce false alarms
- Data Balance & Case Difficulty
 - Build a “difficult cases” subset for targeted training or curriculum learning

Thank you!
